

Trees and model averaging

Audrey Poterie

Office B137 building Yves Coppens

audrey.poterie@univ-ubs.fr

2026-2027 - M2 DSMS, M2 SDD

Chap. 2 : Decision trees

2.1 Introduction

- ▷ **Supervised** learning methods.
- ▷ **Popular methods** that have been applied in many fields such as, for example, electrical engineering, biology, medical research and financial topics.
- ▷ **Methods with a wide applicability**: they can be applied to a wide range of data.
 - ➔ Trees can handle both continuous and categorical inputs.
 - ➔ Methods generally can deal with large n and/or large p (large datasets).

- ▷ **Simple methods:** easy to implement and understand.
 - ▷ Methods that can be easily interpreted thanks to **visualization tools**: trees can be displayed graphically.
 - ▷ Existence of **multiple decision tree algorithms**, such for instance: CART, CHAID, C4.5, CARTGV., etc.
 - ↪ See for example the handbook for a review of the tree-based methods [[Rokach and Maimon, 2005](#)]).
- **Focus on the CART algorithm** [[Breiman et al., 1984](#)] which is one of the most widely used tree algorithms.

2.2 CART principle

CART principle

- ▷ **CART** for **C**lassification **A**nd **R**egression **T**ree.
 - An algorithm in supervised learning for **both regression and classification problems**
- ▷ Method introduced by Leo Breiman (1984).
- ▷ Very famous algorithm : part of the top 10 data mining algorithms identified by the IEEE International Conference on Data Mining (ICDM) in December 2006 [Wu et al., 2008].
- ▷ **Use as building blocks** in lots of powerfull supervised learning methods.
- ▷ **Non-parametric method**: no assumption required for the couple $(\mathbf{X}, Y)$.
 - No need to check some "validity" conditions/hypothesis such as :
 - Linear relationship between $\mathbf{X}$ and Y ?
 - Correlation or collinearity between predictors ?
 - Gaussian distribution for $\mathbf{X}$ and/or Y ?
 - *etc.*

Notations and context: the "classical" supervised learning framework with

- ▷ $Y \in \mathcal{Y}$ = the response variable, the **output**.
- ▷ $\mathbf{X} = (X_1, \dots, X_p) \in \mathcal{X} \subseteq \mathbb{R}^p$ = the vector of p predictors, the **inputs**.
- ▷ Y can be a real ($\mathcal{Y} \subseteq \mathbb{R}$) or a label ($\mathcal{Y} = \{1, \dots, K\}, K \in \mathbb{N}^+$).
- ▷ $X_1, \dots, X_p$ can be continuous, categorical or both.
- ▷ p could be large.

Problem (the objective in supervised learning): **explain/predict** the response $Y \in \mathcal{Y}$ by the vector $\mathbf{X} \in \mathcal{X}$.

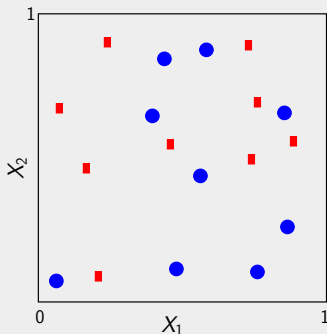
CART's principle:

Build prediction rules by means of **recursive and binary splits** of the predictor space $\mathcal{X}$.

→ Let's start with a toy example: a simple problem in binary classification.

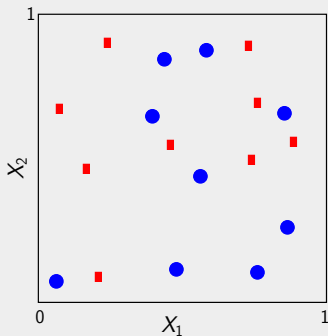
A toy example in classification

Data: a sample $\mathcal{D}_n = \{(\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_n, Y_n)\}$ with $\mathbf{X}_i \in [0, 1]^2$ and $Y_i \in \{\blacksquare, \bullet\}$.



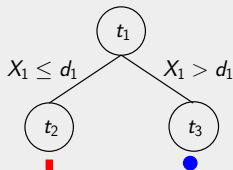
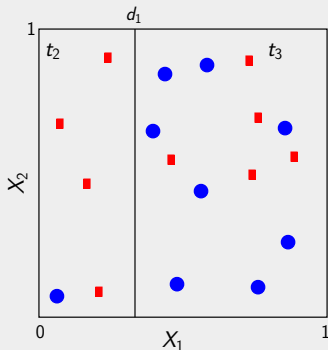
- ▷ **Objective**: well discriminate blues dots and red squares.
- ▷ **CART principle**: build a partition of the feature space that discriminates "at best" the blue dots and the red squares.

A toy example in classification



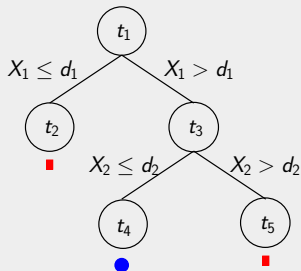
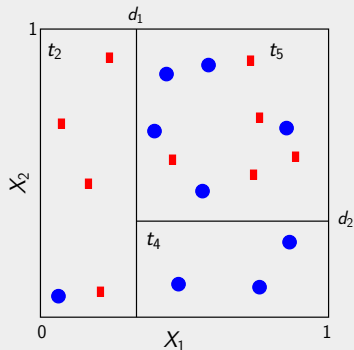
- ▷ Use recursive and binary splits of the feature space.
- ▷ Splits parallel to the axes.
- ▷ A split = choice of a **split variable** and a **split threshold**.

A toy example in classification



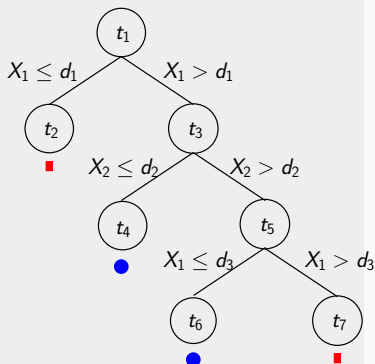
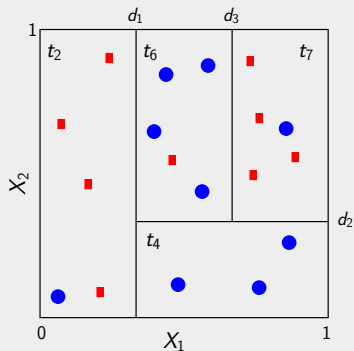
- ▷ At each step, the method splits the data into two regions (or nodes) according to a **split variable** and a **split threshold**.
- ▷ The partition can be represented by means of a tree.

A toy example in classification



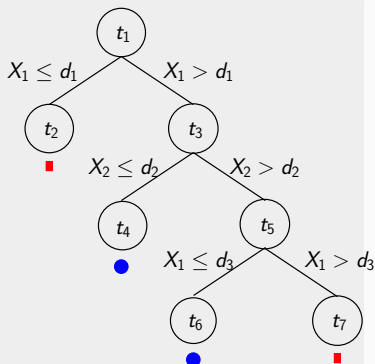
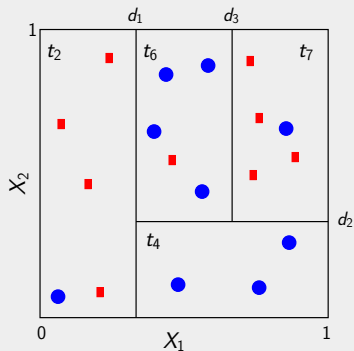
- ▷ At each step, the method splits the data into two regions (or nodes) according to a **split variable** and a **split threshold**.
- ▷ The partition can be represented by means of a tree.

A toy example in classification



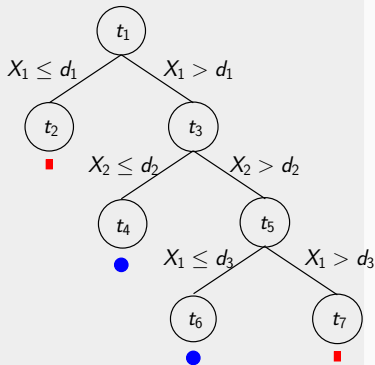
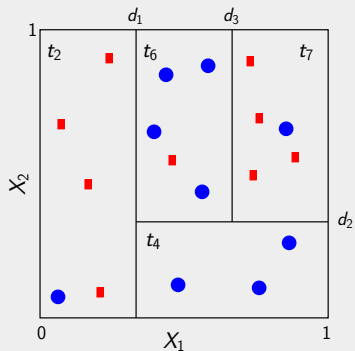
- ▷ At each step, the method splits the data into two regions (or nodes) according to a **split variable** and a **split threshold**.
- ▷ The partition can be represented by means of a tree.

A toy example in classification



- ▷ The final tree T_n is a partition $\mathcal{P}$ of data space made of the terminal nodes (or the leaves) of the tree.
- ▷ In the example above, $\mathcal{P}$ is made of 4 leaves and 3 internal nodes.

A toy example in classification

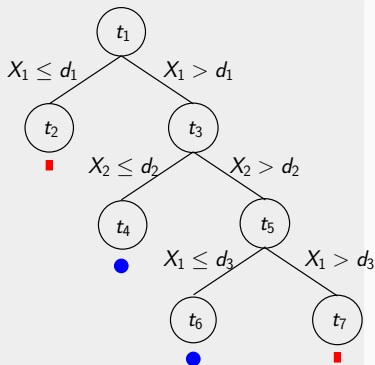
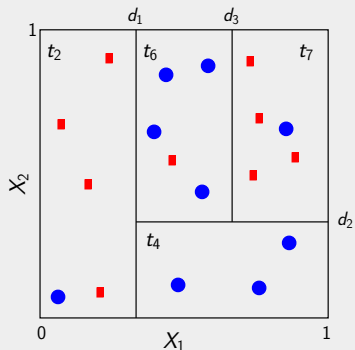


- ▷ In classification: at the end, we do a **majority vote** for Y in each region of the final partition.
- ▷ In regression: at the end, we do the **mean** for Y in each region of the final partition.

2.3 Vocabulary

- ▷ Each split defines **two child nodes**: the left and right child nodes.
- ▷ The terminal nodes (or the leaves) = the regions of the final partition.
- ▷ The **root** = the first node (= the whole feature space $\mathcal{X}$).

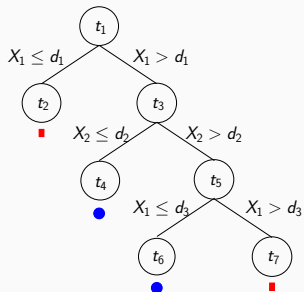
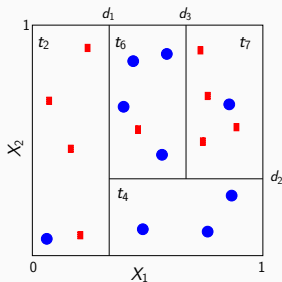
A toy example in classification



- ▷ t_1 is the root.
- ▷ t_4 is the left child node of t_3 and t_3 is the right child node of t_1 .
- ▷ In the tree above, the partition $\mathcal{P} = \{t_2, t_4, t_6, t_7\}$.

2.4 Prediction rule of a tree

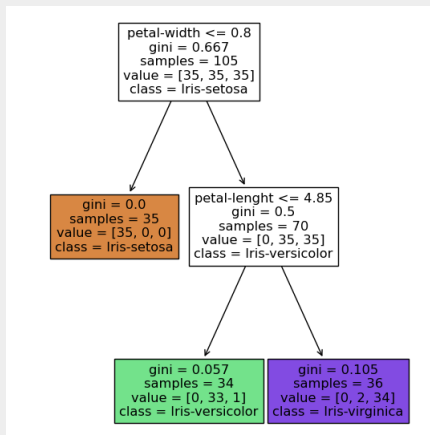
Prediction rule of a tree



At the end,

- ▷ In classification: do the **majority vote** for Y in each region of the final partition.
- ▷ In regression: do the **mean** for Y in each region of the final partition.

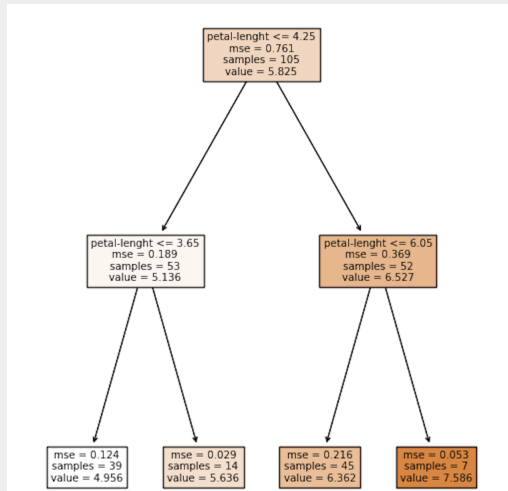
Predict the class of iris flower



A classification tree of depth 2.

- ▷ The first split is defined according the variable `petal-width` and the threshold 0.8.
- ▷ There are 105 observations in the root and 35 in the left root's childnode.
- ▷ There are 3 leaves and each leaf predicts a kind of iris.

Predict the sepal length of iris flower



A regression tree of depth 2.

- There are 4 leaves and predictions in leaves are (from the left to the right): 4.956, 5.363, 6.362, 7.586.

For any observation $\mathbf{x} \in \mathbb{R}^p$, the prediction function of a T_n can be defined as

$$f_n(\mathbf{x}) = \sum_{t \in \mathcal{P}} c_t \cdot \mathbb{1}_{\mathbf{x} \in t}.$$

where

- ▷ The final tree T_n = a partition $\mathcal{P}$ of the feature space $\mathbb{R}^d$.
- ▷ The partition $\mathcal{P}$ = the set made of the leaves (or terminal nodes).
- ▷ $\mathbb{1}_{\mathbf{x} \in t}$ = an indicator function that indicates if the observation $\mathbf{x}$ falls into the node t . Formally, $\mathbb{1}_{\mathbf{x} \in t}$ is defined as

$$\begin{cases} \mathbb{1}_{\mathbf{x} \in t} = 1 & \text{if } \mathbf{x} \in t \\ \mathbb{1}_{\mathbf{x} \in t} = 0 & \text{if } \mathbf{x} \notin t. \end{cases}$$

- ▷ c_t = the prediction in node t .
→ All observations that fall into the node t are predicted as c_t .

For a tree T_n and any observation $\mathbf{x} \in \mathbb{R}^p$, the prediction function of a T_n can be defined as

$$f_n(\mathbf{x}) = \sum_{t \in \mathcal{P}} c_t \mathbb{1}_{\mathbf{x} \in t}.$$

- ▷ In regression, the prediction in each node is the **mean** of Y in the node. Formally, in node t , the prediction $\hat{c}_t$ is defined as

$$c_t = \bar{y}_t = \frac{1}{n_t} \sum_{i: \mathbf{x}_i \in t} y_i.$$

- ▷ In classification, the prediction in each node is the **majority class** of Y in the node. Formally, in node t , the prediction c_t is defined as

$$c_t = \operatorname{argmax}_{k \in \{1, \dots, K\}} \sum_{i: \mathbf{x}_i \in t} \mathbb{1}_{y_i = k},$$

where $k \in \{1, \dots, K\}$ are the labels (or classes) of Y .

2.5 Performance of a Tree

Performance of a Tree

To measure the performance of a tree T_n , we use the empirical risk estimated on $\mathcal{T} = \{(\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_m, Y_m)\}$ and defined as

- ▷ **Example in regression:** the empirical MSE, that is

$$\mathcal{R}_{emp}(f_n) = \frac{1}{m} \sum_{i=1}^m (Y_i - f_n(\mathbf{X}_i))^2.$$

- ▷ **Example in classification:** the empirical misclassification error, that is

$$\mathcal{R}_{emp}(f_n) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}_{Y_i \neq f_n(\mathbf{X}_i)}.$$

Remarks:

- Ideally, $\mathcal{T}$ is a dataset from the same population of $\mathcal{D}_n$ but independent from $\mathcal{D}_n$.
- Other performance metrics can be used (see chapter 1).

Performance of a Tree

To measure the performance of a tree T_n , we use the empirical risk estimated on $\mathcal{T} = \{(\mathbf{X}_1, Y_1), \dots, (\mathbf{X}_m, Y_m)\}$ and defined as

- ▷ **Example in regression:** the empirical MSE, that is

$$\mathcal{R}_{emp}(f_n) = \frac{1}{m} \sum_{i=1}^m (Y_i - f_n(\mathbf{X}_i))^2.$$

- ▷ **Example in classification:** the empirical misclassification error, that is

$$\mathcal{R}_{emp}(f_n) = \frac{1}{m} \sum_{i=1}^m \mathbb{1}_{Y_i \neq f_n(\mathbf{X}_i)}.$$

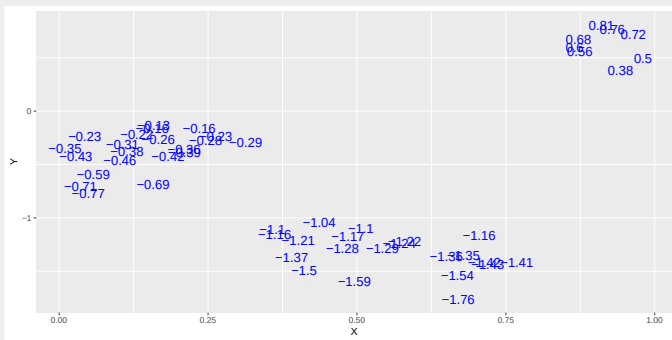
Remarks:

- Ideally, $\mathcal{T}$ is a dataset from the same population of $\mathcal{D}_n$ but independent from $\mathcal{D}_n$.
- Other performance metrics can be used (see chapter 1).

A toy example in regression

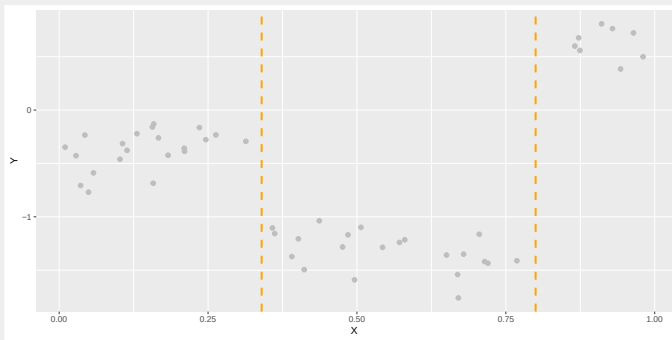
Problem: explain a continuous variable Y based on one predictor X .

Data: a sample $\mathcal{D}_n = \{(\mathbf{X}_1, Y_1), \dots (\mathbf{X}_n, Y_n)\}$ with $X_i \in [0, 1]$, $Y_i \in [-2, 1]$ and $n = 50$.



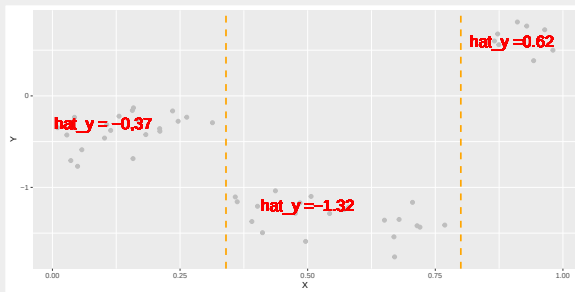
The data

A toy example in regression

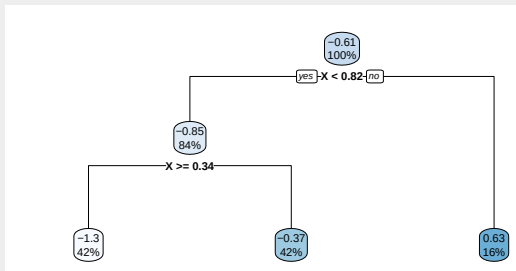


A partition of the data obtained with the CART algorithm

A toy example in regression



A partition of the data obtained with the CART algorithm



The associated CART tree

2.6 Choice of a split

To build a CART tree, we need to answer to these **two main questions**:

Question 1: How to choose an *"optimal"* split ?

Question 2: When to stop splitting ?

CART's principle:

Build prediction rules by means of **recursive and binary splits** of the predictor space $\mathcal{X}$.

The splitting process:

At each step, we split a node t into two child nodes (*Left* and *Right*) w.r.t a **split variable** X_j and a **split threshold** d :

$$t_L(j, d) = \{\mathbf{X} \in t | X_j \leq d\} \quad \text{and} \quad t_R(j, d) = \{\mathbf{X} \in t | X_j > d\}.$$

→ Constraint: we want that child nodes are as **pure** as possible (pure = homogeneous for Y).

→ Conclusion: cut t = find a couple (j, d) minimizing **an impurity criterion**.

CART's principle:

Build prediction rules by means of **recursive and binary splits** of the predictor space $\mathcal{X}$.

The splitting process:

At each step, we split a node t into two child nodes (*Left* and *Right*) w.r.t a **split variable** X_j and a **split threshold** d :

$$t_L(j, d) = \{\mathbf{X} \in t | X_j \leq d\} \quad \text{and} \quad t_R(j, d) = \{\mathbf{X} \in t | X_j > d\}.$$

→ Constraint: we want that child nodes are as **pure** as possible (pure = homogeneous for Y).

→ Conclusion: cut t = find a couple (j, d) minimizing **an impurity criterion**.

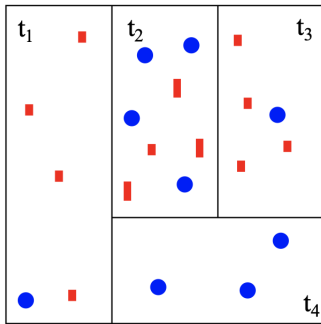
Definition: impurity function

An impurity function measures the homogeneity of the output variable Y within a set (here a node).

For a node, the impurity is:

- **Small** when the node is **homogeneous** (according to Y): values of Y are close to each other in the node.
- **Large** when the node is **heterogeneous** (according to Y): values of Y are different from each other in the node.

Impurity: illustration



- t_4 is pure: impurity in t_4 is very small (equals 0).
- t_2 is less pure than the other nodes: impurity in t_2 is larger.

The splitting process:

At each step, we split a node t into two child nodes (*Left* and *Right*) w.r.t a split variable X_j and a split threshold d :

$$t_L(j, d) = \{\mathbf{X} \in t | X_j \leq d\} \quad \text{and} \quad t_R(j, d) = \{\mathbf{X} \in t | X_j > d\}.$$

→ Conclusion: cut t = find a couple (j, d) minimizing an impurity criterion.

→ Constraint: *we want child nodes as pure as possible.*

→ How to find the optimal split ?

The splitting optimization problem:

We want to find the optimal couple (j^*, d^*) that **maximizes the difference between the impurity in the parent node and the impurity in the child node** (i.e. the impurity decrease). So formally, we seek the couple (j, d) that maximizes

$$I(t) - [p_{t_L(j,d)} I(t_L(j,d)) + p_{t_R(j,d)} I(t_R(j,d))], \quad (1)$$

where

- $I(t)$: the impurity in node t ,
- $I(t_s(j,d))$: the impurity in node $t_s(j,d)$, for $s \in \{L, R\}$,
- N_t : the number of observations in t ,
- $p_{t_s(j,d)} = \frac{N_{t_s(j,d)}}{N_t}$: the proportion of observations in t that fall into $t_s(j,d)$, for $s \in \{L, R\}$.

→ There exist **several impurity functions**: it depends if it's a classification problem or a regression one.

2.7 Choice of a split in regression

In regression: Y is continuous.

→ Usually, the impurity function used I is the variance in the node defined as

$$I(t) = \frac{1}{N_t} \sum_{i: \mathbf{x}_i \in t} (Y_i - \bar{Y}_t)^2,$$

with

- N_t : the number of observations into t ,
- $\bar{Y}_t = \frac{1}{N_t} \sum_{i: \mathbf{x}_i \in t} Y_i$: the mean of Y in the node t .

We want to find the couple (j^*, d^*) that **maximizes the difference between the variance in the parent node and the variance in the child node**. So formally, we seek the couple (j, d) that maximizes

$$\underbrace{\frac{1}{N_t} \sum_{i: \mathbf{X}_i \in t} (Y_i - \bar{Y}_t)^2}_{I(t)} - \left[\underbrace{p_{t_L} \left(\frac{1}{N_{t_L}} \sum_{i: \mathbf{X}_i \in t_L} (Y_i - \bar{Y}_{t_L})^2 \right)}_{I(t_L)} + \underbrace{p_{t_R} \left(\frac{1}{N_{t_R}} \sum_{i: \mathbf{X}_i \in t_R} (Y_i - \bar{Y}_{t_R})^2 \right)}_{I(t_R)} \right],$$

where

- $\bar{Y}_t$: the mean of Y in node t ,
- $\bar{Y}_{t_s}$: the mean of Y in $t_s(j, d)$, for $s \in \{L, R\}$,
- N_t : the number of observations in t ,
- $p_{t_s} = \frac{N_{t_s}}{N_t}$: the proportion of observations in t that fall into $t_s(j, d)$, for $s \in \{L, R\}$.

Remark: for sake of simplicity, we use t_s instead of $t_s(j, d)$ (for $s \in \{L, R\}$) but remind that $t_s(j, d)$ is defined completely according to t , and the couple (j, d) .

We want to find the couple (j^*, d^*) that **maximizes the difference between the variance in the parent node and the variance in the child node**. So formally, we seek the couple (j, d) that maximizes (after reduction)

$$\underbrace{\sum_{i:\mathbf{X}_i \in t} (Y_i - \bar{Y}_t)^2}_{\mathbf{I}(t)} - \left[\underbrace{\sum_{i:\mathbf{X}_i \in t_L} (Y_i - \bar{Y}_{t_L})^2}_{\hat{p}_{t_L} \mathbf{I}(t_L)} + \underbrace{\sum_{i:\mathbf{X}_i \in t_R} (Y_i - \bar{Y}_{t_R})^2}_{\hat{p}_{t_R} \mathbf{I}(t_R)} \right],$$

where

- $\bar{Y}_t$: the mean of Y in node t ,
- $\bar{Y}_{t_s}$: the mean of Y in $t_s(j, d)$, for $s \in \{L, R\}$,
- N_t : the number of observations in t ,
- $p_{t_s} = \frac{N_{t_s}}{N_t}$: the proportion of observations in t that fall into $t_s(j, d)$, for $s \in \{L, R\}$.

Remark: for sake of simplicity, we use t_s instead of $t_s(j, d)$ (for $s \in \{L, R\}$) but remind that $t_s(j, d)$ is defined completely according to t , and the couple (j, d) .

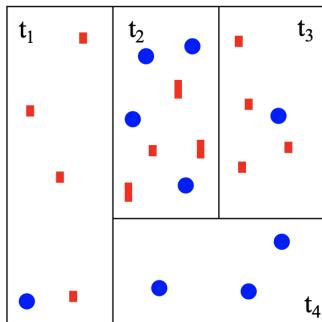
2.8 Choice of a split in classification

In classification: Y is a label, $Y \in \{1, \dots, K\}$.

The impurity measure I should be:

- **Small** when the node is **homogeneous** for Y : one label of Y appears more frequently in t compared to the others.
→ t pur $\Leftrightarrow$ only one unique label in $t \Leftrightarrow I(t) = 0$.
- **Large** when the node is **heterogeneous** for Y : there is no "majority" label of Y in t .

Impurity: illustration



- t_4 is pure: impurity in t_4 is very small (equals 0).
- t_2 is less pure than the other nodes: impurity in t_2 is larger.

Notation: the proportion of observations with label k in t

$$p_{kt} = \frac{1}{N_t} \sum_{i: \mathbf{X}_i \in t} \mathbb{1}_{\{Y_i=k\}}, \quad k = 1, \dots, K,$$

with $\mathbb{1}_{\{Y_i=k\}} = \begin{cases} 1 & \text{if } Y_i = k, \\ 0 & \text{otherwise.} \end{cases}$

Impurity functions usually used in classification:

- **Gini impurity**: $I(t) = \sum_{k=1}^K p_{kt}(1 - p_{kt})$.
- **Cross-entropy**: $I(t) = -\sum_{k=1}^K p_{kt} \log(p_{kt})$. where p_{kt} denotes the proportion of observations with label k in t , for $k = 1, \dots, K$.

Remark: CART uses Gini by default.

Notation: the proportion of observations with label k in t

$$p_{kt} = \frac{1}{N_t} \sum_{i: \mathbf{X}_i \in t} \mathbb{1}_{\{Y_i=k\}}, \quad k = 1, \dots, K,$$

with $\mathbb{1}_{\{Y_i=k\}} = \begin{cases} 1 & \text{if } Y_i = k, \\ 0 & \text{otherwise.} \end{cases}$

Impurity functions usually used in classification:

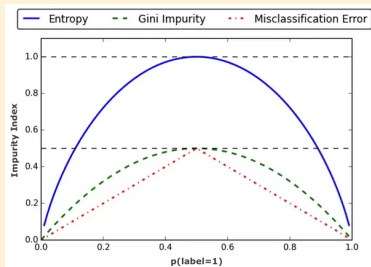
- **Gini impurity:** $I(t) = \sum_{k=1}^K p_{kt}(1 - p_{kt})$.
- **Cross-entropy:** $I(t) = -\sum_{k=1}^K p_{kt} \log(p_{kt})$. where p_{kt} denotes the proportion of observations with label k in t , for $k = 1, \dots, K$.

Remark: CART uses Gini by default.

In Binary classification: only two labels, i.e. $Y = 0$ or 1 .

- **Gini impurity:** $I(t) = 2p_{1t}(1 - p_{1t})$,
- **Cross-entropy:** $I(t) = -p_{1t} \log(p_{1t}) - p_{0t} \log(p_{0t})$,

where p_{kt} denotes the proportion of observations with label k in t , for $k = 0, 1$.



Impurity functions for binary classification

Remark: for impurity function, choose preferably Gini or equivalently cross-entropy and avoid misclassification error.

We want to find the couple (j^*, d^*) that maximizes the difference between the impurity in the parent node and the impurity in the child node. So formally, we seek the couple (j, d) that maximizes

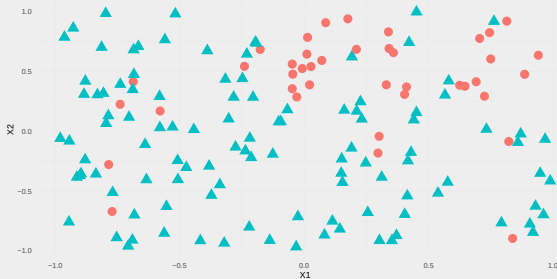
$$\underbrace{\sum_{k=1}^K p_{kt}(1 - p_{kt})}_{I(t)} - \left[\underbrace{p_{t_L} \sum_{k=1}^K p_{kt_L}(1 - \hat{p}_{kt_L})}_{p_{t_L} I(t_L)} + \underbrace{p_{t_R} \sum_{k=1}^K p_{kt_R}(1 - p_{kt_R})}_{p_{t_R} I(t_R)} \right],$$

where

- K : the number of labels for Y ,
- p_{kt} denotes the proportion of observations with label k in t , for $k = 1, \dots, K$,
- p_{kt_s} denotes the proportion of observations with label k in t_s , for $k = 1, \dots, K$ and $s \in \{L, R\}$.

Remark: for sake of simplicity, we use t_s instead of $t_s(j, d)$ (for $s \in \{L, R\}$) but remind that $t_s(j, d)$ is defined completely according to t , and the couple (j, d) .

Choice of a split: example (in classification)



Impurity estimated on the data $I(t) = 0.3942$ (initial impurity).

Choice of a split: example (in classification)

Two split candidates



- Split 1: split variable = x_2 and the threshold = 0.29,
- Split 2: split variable = x_1 and the threshold = 0.058.

→ What split do we choose ?

Choice of a split: example (in classification)



	$\mathcal{I}(t_L)$	$\mathcal{I}(t_R)$	$\hat{p}_{t_L}$	$\hat{p}_{t_R}$	Impurity gain
Split 1	0.4978	0.1619	0.4	0.6	0.094
Split 2	0.4898	0.1734	0.51	0.49	0.054

→ What split do we choose ? Split 1.

Choice of a split: example (in classification)



	$\mathcal{I}(t_L)$	$\mathcal{I}(t_R)$	$\hat{p}_{t_L}$	$\hat{p}_{t_R}$	Impurity gain
Split 1	0.4978	0.1619	0.4	0.6	0.094
Split 2	0.4898	0.1734	0.51	0.49	0.054

→ What split do we choose ? **Split 1.**

2.9 Pruning

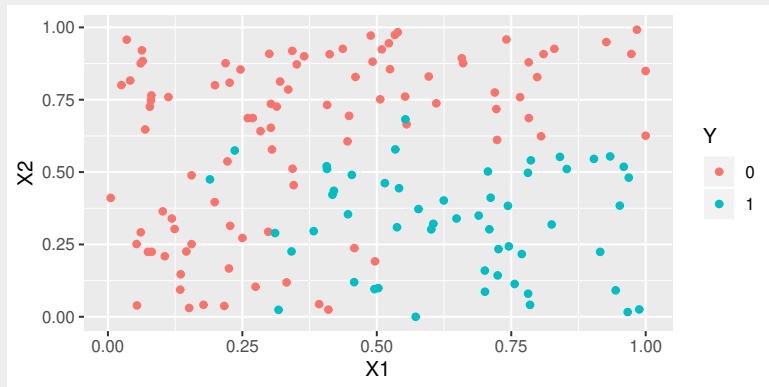
To build a CART, we need to answer to **two main questions**:

1 How to choose an *"optimal"* split ?

2 When to stop splitting ?

Illustration

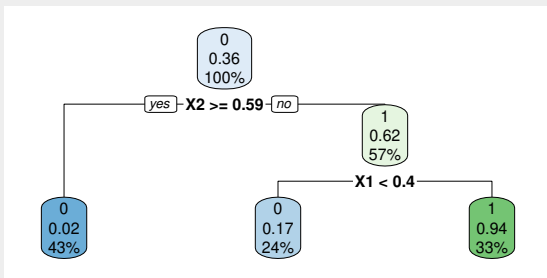
Data: a simple binary classification problem with 150 observations, two inputs X_1 and X_2 .



→ Here, after looking at the data representation, we might want to split twice.

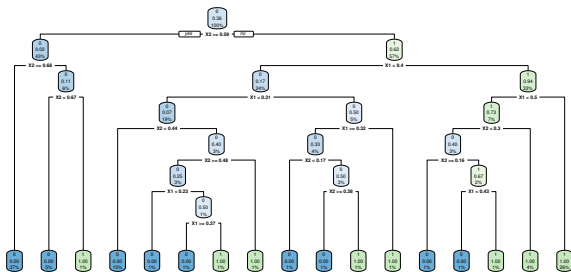
Illustration

The associated CART tree with only two splits:



Illustration

The maximum CART tree with 16 splits:



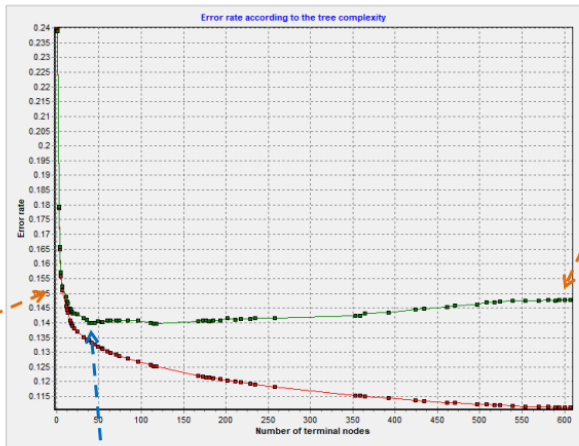
Illustration

Comparison between the two trees:

	Misclassification error**	Number of splits
Small tree	0.31	2
Maximal tree	0.37	16

***Misclassification error computed using 10-fold cross validation.*

→ Performance is not always improved by the size of the tree.



Underfitting: the model performs poorly because it is too simple to capture the underlying relationship between the inputs (features) and the response (target).

Overfitting: the tree explains almost perfectly the training data but performs poorly on test data

The optimal size of the tree. How to find it ?

There is a dilemma:

- ▷ To choose too small/simple tree = very robust/stable model (small variance)... but does not fit well the data (large bias) .
↪ It is what we call **underfitting**.
- ▷ To choose too large/complex tree = fits perfectly the data (low bias)... but unstable model (large variance)
↪ It is what we call **overfitting**.

⇒ We need to find a trade-off.

There exist two types of strategies:

- ▷ **Pre-pruning strategy:** we define stopping rules before the tree building and then we stop splitting when one of the stopping conditions is satisfied.
- ▷ **Post-pruning strategy:** we build the largest tree and after we select the final tree by removing some branches.

Pre-pruning strategy = at the beginning, we define stopping rules and then we stop splitting when one of the stopping conditions is satisfied.

Stopping rules can be for instance:

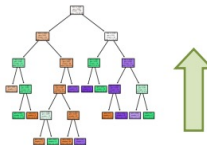
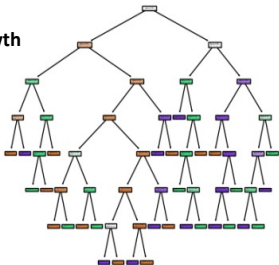
- ▷ a maximal depth,
- ▷ a minimum node size,
- ▷ a minimum gain in impurity.

→ Several stopping rules can be used at the same time. In this case, **we stop growing the tree as soon as one stopping rule is satisfied.**

Post-pruning

Post-pruning strategy = we build the largest tree and after we select the final tree by removing some "irrelevant" subbranches of the tree starting from the bottom.

Tree-growth
phase



Post-pruning: remove iteratively irrelevant (according to a criterion) subbranches starting from the bottom

Cost-Complexity Pruning (Breiman / CART)

1. **Grow the maximal tree:** build a highly complex tree with **low bias** but **high variance**.
2. **Build a sequence of optimal trees** by using the **cost-complexity criterion**.
3. **Select the final tree:** use **cross-validation** (or a validation set) to choose the value of α (and hence the corresponding subtree), typically minimizing the validation error.

The cost-complexity criterion (Breiman, 1984)

Let define the **cost complexity criterion** of the tree T as

$$\mathcal{R}_\alpha(T) = R(T) + \alpha|\tilde{T}|,$$

where

- $R(T)$: a quality criterion, the error of the tree is often used.
- $|\tilde{T}|$ = the number of leaves of T ,
- $\alpha \in \mathbb{R}^+$ = a regularization parameter **which controls the bias-variance tradeoff.**

We need to choose a value for α :

- if $\alpha = 0 \implies$ Choose the deepest tree denoted by $T_{\max}$.
- if $\alpha = +\infty \implies$ Choose the tree with no split, which corresponds to the root of the tree denoted by t_1 .

→ How to find an "optimal" value for α ?

Fortunately, there is no need to try all real values for α . Indeed, the following theorem tells that **there exists a finite and unique sequence of relevant values for α .**

Theorem (Breiman, 1984)

For any non-trivial and tree T built on $\mathcal{D}_n$ and with root t_1 , there exist a **unique sequence**

$$0 = \alpha_1 < \dots < \alpha_K = \infty$$

and a **unique sequence of nested subtrees** of T

$$T \supseteq T_1 \supset \dots \supset T_K = \{t_1\}$$

such that for every $1 \leq k < K$,

$$\forall \alpha \in [\alpha_k; \alpha_{k+1}[, \quad T_k = \operatorname{argmin}_{T' \preceq T_n} \mathcal{R}_\alpha(T'), \text{ and}$$

$$\forall \alpha \geq \alpha_K, \quad T_K = \operatorname{argmin}_{T' \preceq T_n} \mathcal{R}_\alpha(T').$$

Fortunately, there is no need to try all real values for α . Indeed, the following theorem tells that **there exists a finite and unique sequence of relevant values for α .**

Theorem (Breiman, 1984)

For any non-trivial and tree T built on $\mathcal{D}_n$ and with root t_1 , there exist a **unique sequence**

$$0 = \alpha_1 < \dots < \alpha_K = \infty$$

and a **unique sequence of nested subtrees** of T

$$T \supseteq T_1 \supset \dots \supset T_K = \{t_1\}$$

such that for every $1 \leq k < K$,

$$\forall \alpha \in [\alpha_k; \alpha_{k+1}[, \quad T_k = \operatorname{argmin}_{T' \preceq T_n} \mathcal{R}_\alpha(T'), \text{ and}$$

$$\forall \alpha \geq \alpha_K, \quad T_K = \operatorname{argmin}_{T' \preceq T_n} \mathcal{R}_\alpha(T').$$

- The sequence of optimal subtrees for increasing values of α can be found **iteratively and efficiently**.
- Breiman does not test all possible subtrees: he iteratively prunes the **“weakest links”** (i.e., the subtrees whose removal causes the smallest increase in cost per leaf removed), thereby efficiently constructing a finite sequence of **nested optimal subtrees**.
- In other words, instead of enumerating all possible subtrees, Breiman’s algorithm uses a **successive pruning strategy**: at each step, it removes the subtree with the smallest *increase in cost per leaf removed*. This directly produces the **sequence of nested optimal subtrees**.

 $\alpha = 0$

The largest tree with 5 leaves

 $\alpha = 000716$

The tree with 4 leaves

 $\alpha = 0,02442$

The tree with 3 leaves


$$\alpha = 0,025032$$

The tree with 2 leaves

 $\alpha = 0,31531$

The root

Which subtree do we select ?

→ Use crossvalidation or a validation set.

One Standard-Error Rule (1-SE rule)

- Choosing the “optimal” tree based on a pruning set or cross-validation can make the selected tree **sensitive to the particular sample or cross-validation splits**.
- **Suggested solution:** choose the smallest tree whose estimated error is not significantly larger than the minimum estimated error. → This is called the **1-SE rule**.

One Standard-Error Rule (1-SE rule)

Let R^* be the minimum estimated error among the candidate subtrees, and let σ^* be its standard error. The one-SE rule selects the **smallest subtree** whose estimated error R satisfies

$$R \leq R^* + \sigma^*.$$

One Standard-Error Rule (1-SE rule)

- Choosing the “optimal” tree based on a pruning set or cross-validation can make the selected tree **sensitive to the particular sample or cross-validation splits**.
- **Suggested solution:** choose the smallest tree whose estimated error is not significantly larger than the minimum estimated error. → This is called the **1-SE rule**.

One Standard-Error Rule (1-SE rule)

Let R^* be the minimum estimated error among the candidate subtrees, and let σ^* be its standard error. The one-SE rule selects the **smallest subtree** whose estimated error R satisfies

$$R \leq R^* + \sigma^*.$$

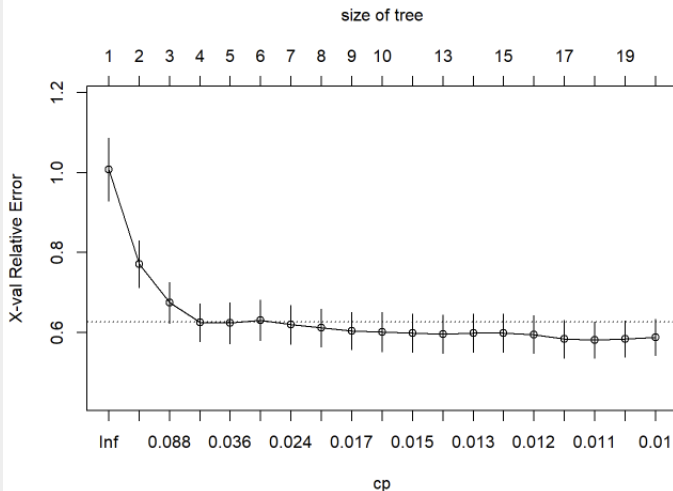
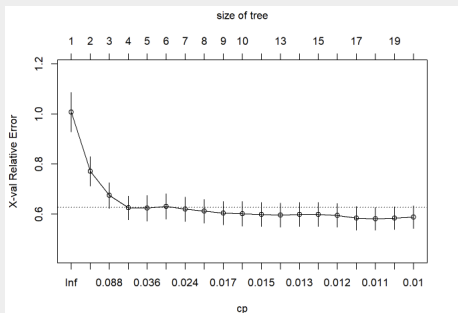


Figure 1: Plot illustrating the relative cross validation error (y-axis) for various α values (cp in x-axis). Tree size indicates the number of leaves. The dotted line is the line with equation $y = R^* + 1 \times \sigma^*$.

Illustration



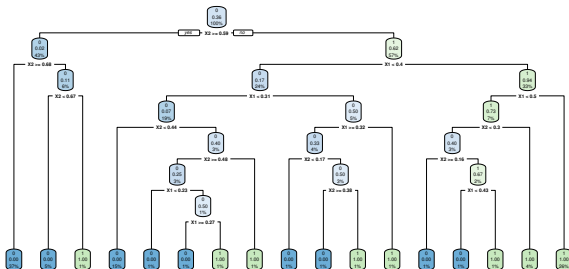
Interpretation:

- Smaller α values lead to larger trees.
- Minimal relative cross validation error is achieved for the tree with 18 terminal nodes.
- Using the 1-SE rule, we select the tree with 7 terminal nodes.

Example: step 1 - grow the largest tree

```
1 # R command
2 library(rpart)
3 library(rpart.plot)
4 tree1 <- rpart(Y~.,data=data1,cp=0.01,minsplit=2)
5 rpart.plot(tree1)
6
```

Output:



The deepest tree, $T_{\max}$ ($=T_0$)

Example: step 3 - choice of the final tree

Implementation of the cost-complexity pruning in R

```
1 #R command
2 printcp(tree1)
3
```

Output:

Classification tree: rpart(formula = Y ~ ., data = data)

Variables actually used in tree construction:

[1] X1 X2

Root node error: 54/150 = 0.36

n= 150

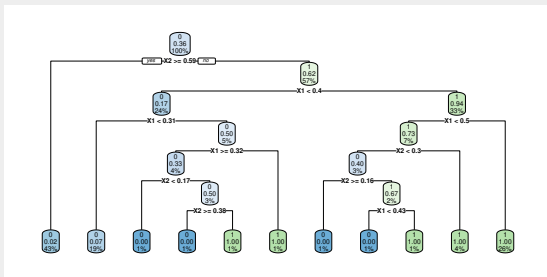
CP	nsplit	rel error	xerror	xstd
0.407407	0	1.000000	1.00000	0.108866
0.018519	2	0.185185	0.25926	0.065977
0.013889	6	0.111111	0.37037	0.077099
0.010000	10	0.055556	0.35185	0.075435

→ We select the tree minimizing the empirical risk estimate computed by KF cross validation (denoted `xerror` in the R output) : so we choose T_3 .

Example: the sequence of optimal subtrees

```
#R command  
T1 <- prune(tree1,cp=0.01)  
rpart.plot(T1)
```

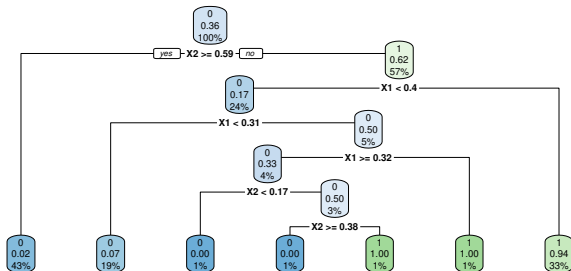
Output:



Example: the sequence of optimal subtrees

```
#R command  
T2 <- prune(tree1, cp=0.013889)  
rpart.plot(T2)
```

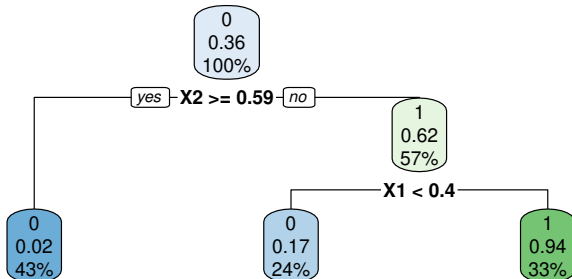
Output:



Example: the sequence of optimal subtrees

```
1 #R command  
2 T3 <- prune(tree1, cp=0.018519)  
3 rpart.plot(T3)  
4
```

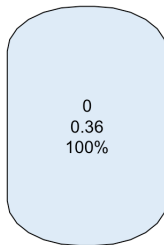
Output:



Example: the sequence of optimal subtrees

```
1  #R command
2  T4 <- prune(tree1,cp=0.407407)
3  rpart.plot(T4)
4
```

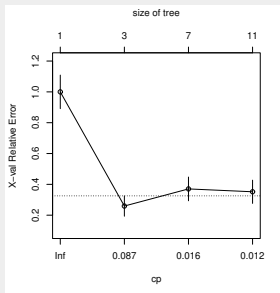
Output:



Example: step 3 - choice of the final tree

```
#R command  
plottcp(tree1)
```

Output:

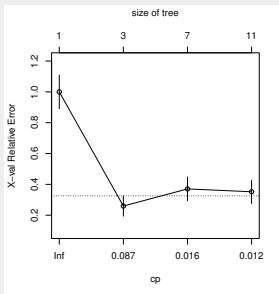


→ Use the 1-SE rule: choose the smallest tree with cross-validated error rate less than the $MinError + MinSE$, ($MinError$ = minimum cross-validated error rate, $MinSE$ = its associated standard error).

Example: step 3 - choice of the final tree

```
#R command  
plottcp(tree1)
```

Output:

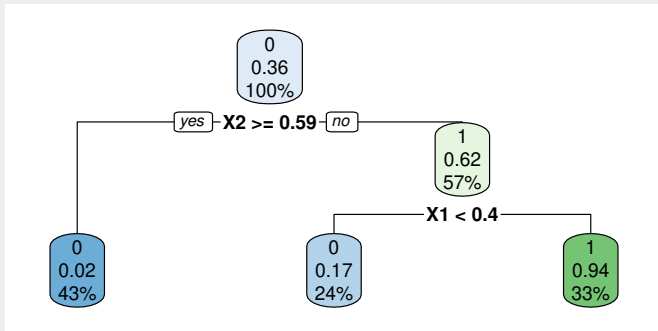


→ Use the **1-SE rule**: choose the smallest tree with cross-validated error rate less than the $MinError + MinSE$, ($MinError$ = minimum cross-validated error rate, $MinSE$ = its associated standard error).

Example: the final tree

```
#R command  
cp_ftree<-tree1$cptable[2,1]  
ftree <- prune(tree1,cp=cp_ftree)  
rpart.plot(ftree)
```

Output:



2.10 Garantie théorique

A First Result

Stone (1977) provides a general framework for L^2 consistency of local averaging estimators,

$$\hat{r}_n(x) = \sum_{i=1}^n W_{n,i}(x) Y_i, \quad \sum_{i=1}^n W_{n,i}(x) = 1,$$

under regularity conditions on the weights and the geometry of the cells.

Breiman et al. (1984) build on this result to establish L^2 consistency results for **partition-based estimators**, under suitable regularity conditions on the partition and provided that

$$k_n \rightarrow \infty \quad \text{and} \quad \frac{k_n}{n} \rightarrow 0,$$

where k_n denotes the **minimum number of observations in each cell**.

Limitation: this result alone does not establish the consistency of the CART algorithm. In particular, it does not prove consistency for the procedure that first grows a maximal tree and then prunes it using cost-complexity pruning.

Theoretical guarantees for CART were established later by Gey & Nédélec (2005) and Gey (2012).

Under suitable regularity conditions:

- **Oracle-type performance:** The empirical procedure (growing the maximal tree, cost-complexity pruning, and then selecting a tree using an independent sample) selects the subtree in the sequence whose risk is, up to a multiplicative constant, of the same order as that of the best subtree of $T_{\max}$.
- **Justification of the penalty:** The analysis of the risk upper bound provides a theoretical justification for the form of the penalty $\alpha|T|$ used in the cost-complexity criterion.

2.11 Conclusion

Key Advantages of Tree-Based Methods

- ✓ Can handle categorical and quantitative response variable.
- ✓ Can handle mixed predictors (categorical and quantitative).
- ✓ Easily ignore redundant variables.
- ✓ Small trees are easy to interpret.
- ✓ Handle cleverly missing data.
- ✓ Low time-complexity: between $\mathcal{O}(np \log n)$ (for the perfect-balanced tree) and $\mathcal{O}(pn^2)$ (for the worst-balanced tree).
- ✓ (Provide an importance index for each predictor).

Limitations of Tree-Based Methods

- ✗ Large trees are not easy to interpret.
 - Sensitive to small changes in the training data.
- ✗ Trees are sensitive to outliers and small changes in the train set.
 - Often lead to lower prediction performance compared to other learning methods.

- **Simple method for both regression and classification:** *easy* to understand, to implement and to interpret.
- **Wide applicability:** non-parametric method, for both categorical and continuous predictors.
- **Main drawback:** algorithm not robust.

→→→ ...But standard model (baseline model) for lots of averaging models such as bagging, random forests, boosting, etc.

- **Simple method for both regression and classification:** *easy* to understand, to implement and to interpret.
- **Wide applicability:** non-parametric method, for both categorical and continuous predictors.
- **Main drawback:** algorithm not robust.
- **...But standard model (baseline model) for lots of averaging models** such as bagging, random forests, boosting, etc.



Breiman, L., Friedman, J., Stone, C. J., and Olshen, R. A. (1984).
Classification and regression trees.
CRC press.



Rokach, L. and Maimon, O. (2005).
Decision trees.
In *Data mining and knowledge discovery handbook*, pages 165–192.
Springer.



Wu, X., Kumar, V., Quinlan, J. R., Ghosh, J., Yang, Q., Motoda, H.,
McLachlan, G. J., Ng, A., Liu, B., Philip, S. Y., et al. (2008).
Top 10 algorithms in data mining.
Knowledge and information systems, 14(1):1–37.

APPENDIX: selection of the final tree by cross-validation when using cost-complexity pruning

Algorithm

1. Find the sequence $0 = \alpha_1 < \dots < \alpha_K = \infty$ and define the geometric midpoint of each interval $[\alpha_k, \alpha_{k+1}[$

$$\beta_k = \sqrt{\alpha_k \alpha_{k+1}}$$

with $\beta_0 = 0$ and $\beta_{K+1} = \infty$

2. Split the data sample $\mathcal{D}_n$ into L blocs $B_1, \dots, B_L$ of size n/L .
3. **For $l=1$ to L , do:**
 - 3.1 Grow the trees $T_{\beta_0}^l, \dots, T_{\beta_{K+1}}^l$ using $\mathcal{D}_n \setminus B_l$.
 - 3.2 For all $i \in B_l$ and all $k = 0, \dots, K+1$, compute the prediction $\hat{y}_i(\beta_k)$ based on $T_{\beta_k}^l$.
4. Compute the risk $\mathcal{R}(k) = \frac{1}{n} \sum_{i=1}^n \mathcal{R}(y_i, \hat{y}_i(\beta_k))$, for all $k = 0, \dots, K+1$.
5. Choose α_{k^*} such that $\beta_{k^*+1} = \operatorname{argmin}_k \mathcal{R}(k)$